

Three architectural decisions

For the Jessica sync — what to add, why, and how much lift each one actually is

None of these are new features. They are decisions about how the features already being built are structured underneath. The goal is to make sure the architecture we ship in V1 doesn't have to be torn up later to build the things that make enrops defensible over time.

01 Event-level data instrumentation

LOW LIFT

- | | |
|------|---|
| WHAT | Don't just store final state (enrolled / cancelled). Log the event trail: when enrollment was initiated, when payment completed, how many days before session start, whether it was a waitlist conversion, cancellation with a reason code. |
| WHY | Final state data (enrolled or not) can be replicated by any competitor. Event trail data accumulated across dozens of operators cannot. The pattern library this creates over time (what programs fill fastest, what timing drives cancellations, what pricing converts) is proprietary to enrops and inaccessible to anyone outside the platform. |
| WHEN | V1. Retrofitting a proper event log into an existing schema is painful. This needs to be built in from the start. |
| HOW | <ul style="list-style-type: none">— Add an <code>enrollment_events</code> table in Supabase: <code>id</code>, <code>operator_id</code>, <code>site_id</code>, <code>session_id</code>, <code>parent_id</code>, <code>action_type</code>, <code>metadata</code> (jsonb), <code>created_at</code>— <code>action_type</code> covers: <code>initiated</code>, <code>payment_completed</code>, <code>waitlist_added</code>, <code>waitlist_converted</code>, <code>cancelled</code> (with reason), <code>refunded</code>— No new UI. Just make sure the registration flow writes to this table at each step alongside whatever it already writes— This is a logging decision, not a feature |

Capacity ask: 2–4 hours to add the table and wire it into existing registration flow logic. No frontend work.

02 Parent accounts at platform level, not operator level

CHECK NOW

- | | |
|------|---|
| WHAT | When a parent creates an account to enroll with Operator A, that account belongs to enrops, not to Operator A. If that same parent later enrolls with Operator B, their history |
|------|---|

travels with them. The parent relationship lives at the platform.

WHY

If parent accounts are operator-scoped, enrops is a tool. If parent accounts are platform-scoped, enrops owns the cross-operator parent relationship. No individual operator's AI agent, and no outside platform, can access that. It's the same thing Shopify did with Shop Pay: the consumer identity layer across the network belongs to the platform.

WHEN

V1 schema decision. Changing this later means a full migration. It has to be right from the start.

HOW

- Confirm: does the parents table have an operator_id foreign key? If yes, that's the problem
- Parents should be a platform-level table with no operator affiliation
- enrollments is the join: it connects a parent to an operator's session
- One parent can have enrollments across multiple operators. That's the network
- Auth should reflect this: parents log into enrops, not into "Operator A's portal"

Capacity ask: this may already be correct. A 15-minute schema check with Jessica is the ask. If it needs adjustment, it's a schema change now rather than a migration later.

03 School partners as platform-level records (not operator contacts)

V1 DATA MODEL, V2 PORTAL

WHAT

School partners should be their own platform-level entities, not contacts nested inside an operator's profile. The V2 school partner portal can be built later, but the data model needs to support a school having relationships with multiple operators from the start.

WHY

If a school coordinator eventually logs into enrops and can see all their enrichment provider relationships across operators, enrops owns the school side of the network. A competitor building from zero has to recruit both operators and schools simultaneously. That's a two-sided rebuild, which is the closest thing to a genuine time moat the ops platform has.

WHEN

Data model in V1, portal in V2. The V1 ask is just the schema. No new UI needed now.

HOW

- Set up school_partners as a platform-level table: id, name, district, address, coordinator_name, coordinator_email, coi_status, facility_notes
- Set up operator_school_partnerships as the join table: operator_id, school_partner_id, status, contract_start, contract_end, notes
- This replaces any structure where school partner data lives only inside an operator's profile
- V2 portal just adds auth for the coordinator_email field that already exists in the table

Capacity ask: schema design only. Probably 1–3 hours to restructure if school partner data is currently operator-nested. The V2 portal builds on top of this and doesn't require revisiting the schema.